Parker Hannifin Future State Architecture (Proposed)

Companion document to parker-current-state-architecture.md. This is a directional target architecture, not a committed roadmap — every component marked "Validation Needed" requires a proof-of-concept or technical spike before it can be treated as a decision.

Drafted 2026-10-10.

Yuji Nakagawa, Expert AEM Solution Consultant



Hybrid Architecture model optimized for business + system agility

This document intentionally does **not** *presuppose that any specific existing property (www, ph, help) moves to EDS.*

The objective is to illustrate how Edge Delivery Services can serve as a FrontEnd on the Edge with multiple pathways for existing headless applications to integrate (side by side or in front, on the edge)

Routing is by business outcome, not per-domain.

This is deliberately not a map of which existing domain (www, ph, help) ends up on which path. The routing decision is made per template or per interaction type, and is expected to differ team-by-team and page-by-page — the architectural point is that the capability to route either way exists, with no duplication of header/footer or content/product data access between the two path.

Shared layers across both rendering paths

Edge / Routing Layer	EDS Rendering Path	React / Headless Path	Identity / SSO
Akamai (or equivalent) Per-route rendering decision — not per-domain / per-property.	**Content-oriented pages** Marketing, editorial, landing pages rendered via Edge Delivery Services.	**Interaction components** Catalog, configurator, account stay React — existing headless investment preserved.	**Okta (Azure AD federated)** Single IdP, used by both rendering paths identically.

Shared Content, Product & Presentation Layer

Content API
Surfaces the same editorial/marketing content to EDS and React identically.

Product / Commerce API
Surfaces the same catalog, pricing, account data to EDS and React identically.

Shared Header / Footer
Single source, federated component — not duplicated per rendering path.

What Changes vs. Current State.

The table below outlines the five target components, what each replaces or evolves, the rationale, and the validation plus proof points required before anything is treated as a settled recommendation.

Target Component	Replaces / Evolves	Rationale	Validation Needed	Proof Point Needed
Edge-level routing between EDS and React/headless paths	Today, rendering approach is fixed per domain (www.parker.com = AEM SPA Editor, ph/help = Next.js)	Decouples "which rendering technology" from "which property" — lets each team/template pick EDS or React based on the actual need (content vs. interaction), without a wholesale platform migration decision	Confirm the edge/CDN layer (Akamai or successor) can route at the template/path level cleanly, and that both paths can coexist under one domain without URL-structure conflicts	Pilot: route two templates on the same property to different paths (e.g. one content page via EDS, one interactive page staying React) and confirm no regressions in SEO, auth, or analytics
Shared Header/Footer across both paths	Today, header/footer logic is embedded per-property (AEM clientlib on www, presumably components within each Next.js app on ph/help)	Avoids maintaining the same nav/footer in two+ places; ensures brand/UX consistency regardless of which rendering path serves a given page	Confirm whether a federated/shared component can be consumed identically by an EDS block and a React component (likely via a small JS web component or shared API-driven data contract, not shared framework code)	Build the header/footer as a single service (e.g. a JSON/HTML fragment API) and consume it from both an EDS block and a React component on a test page
Shared Content & Product Data Layer	Content and product data currently reached differently per property (AEM .model.json for www, presumably a separate headless API for ph/help)	This is the core ask: let the same content/product data surface in either EDS or React, so the rendering-path decision doesn't fork the data model or require content to be authored/maintained twice	Confirm whether existing headless commerce/content APIs (already used by ph/help) can be the single source EDS also reads from — this explicitly preserves the customer IT team's existing headless investment rather than replacing it	Build one EDS block and one React component that both call the same content or product API endpoint and render equivalent output
Identity (Okta) used identically by both paths	Already largely true today — Okta SSO is already shared	Low-risk/no-change item; included for completeness so both rendering paths have one consistent login/account experience	Confirm session/token handling (cookies, redirects) works the same when a user crosses from an EDS-rendered page to a React-rendered page in the same session	Test a login flow starting on an EDS page and completing on a React page (or vice versa), confirm no re-authentication required
⚠️ **Commerce interface layer (not a backend replacement)**	Direct, tightly-coupled calls from front-ends to IBM WebSphere Commerce cached at Adobe Edge Delivery Commerce for performance. (storeId, langId, WCTrustedToken)	An interface/mesh layer between WCS and the front-ends lets both EDS and React consume catalog/pricing/account data through one modern API contract, without touching WCS itself — directly de-risks the shared data-layer goal above by giving it a concrete implementation option	Confirm WCS exposes (or can be made to expose) the raw data these interface layers need — product/catalog/pricing/APIs — likely via custom resolvers/connectors since WCS isn't a natively-supported Adobe Commerce backend	**Adobe Edge Commerce** lightweight GraphQL interface/orchestration layer that can front any backend, including non-Adobe systems like WCS, via custom resolvers. Requires **no additional commerce platform license** — only API Mesh/App Builder, which several Adobe customers already have access to. **Recommended first proof point** given zero incremental licensing cost.

⚠️ **Highlighted row = needs discussion.**

Sequencing Notes (draft, for discussion)

1. **Start with the shared data layer proof point** — build one EDS block and one React component against the same content/product API. This is the foundation the rest depends on, and it's the piece most likely to surface objections from customer IT if the existing headless APIs can't actually serve EDS cleanly.
2. **Shared header/footer can be validated in parallel** — it's lower risk and doesn't depend on the data-layer outcome.
3. **Edge routing pilot** (two templates on one property, split across paths) comes next, once the shared data layer and header/footer are proven — this is where SEO/auth/analytics parity gets tested end-to-end.
4. **Commerce interface layer proof point** can actually run in parallel with step 1, not strictly after it — standing up Adobe Edge Commerce (API Mesh + Catalog Service) in front of WCS as a read-only pilot requires no new license and doesn't touch WCS itself, so it carries little sequencing risk. If it proves out, it becomes the thing the shared data layer (step 1) actually talks to for commerce data. Adobe Commerce Optimizer is a fallback evaluation if Edge Commerce's thinner interface can't meet merchandising/search requirements — evaluate it second, since it carries its own license cost.

Explicit Non-Goals / Open for Debate

- This document does **not** say which of www.parker.com, ph.parker.com, or help.parker.com should move to EDS, stay React, or split. That's a per-template decision made later, informed by the proof points above — not a conclusion of this document.
- This document does **not** ask customer IT to abandon their headless architecture. The shared data layer is framed as EDS becoming *another consumer* of the existing headless APIs, not a replacement for them.
- This document does **not** prescribe replacing WCS as the commerce system of record. It proposes an **interface layer** (Adobe Edge Commerce, with Adobe Commerce Optimizer as a fallback option) in front of WCS — WCS itself stays in place; only the data contract exposed to front-ends changes.